

۰۲ — معماری فنی

اصل راهنما: **کمترین سرویس، بیشترین خودکارسازی**. یک مخزن، یک اپ Next.js که هم سایت است هم پنل مدیریت هم صف کارها، یک پایگاه داده‌ی PostgreSQL.

۱. پشته‌ی فناوری

چرا	انتخاب	لایه
همان پشته‌ی FarsiUI؛ SSR/ISR برای سئو	Next.js 16 (App Router) + React 19 + TypeScript	فرانت و بک‌اند
۶۴ کامپوننت و ۴۸۰ بلاک RTL؛ تقویم شمسی؛ ارقام فارسی	FarsiUI (base-nova) + Tailwind 4	رابط کاربری
پنل ادمین آماده، محلی‌سازی فیلدبه‌فیلد fa/en/ar، احراز هویت، دسترسی‌ها، صف کار داخلی (Jobs Queue) — بدون نوشتن پنل از صفر	Payload CMS 3 (داخل همان اپ Next)	CMS و پنل مدیریت
یک DB برای داده، جست‌وجوی متنی، فازی و معنایی	PostgreSQL 16 + با pg_trgm و pgvector	پایگاه داده
مسیرهای fa/en/ar/ تشخیص زبان، hreflang	next-intl برای رابط + محلی‌سازی Payload برای محتوا	il8n
کار پس‌زمینه بدون سرویس جدا	Payload Jobs Queue (cron داخلی)؛ در صورت رشد → Trigger.dev خودمیزبان	صف و زمان‌بندی
تعویض مدل بدون تغییر کد؛ انطباق با شرایط هر ارائه‌دهنده (سند ۰۹)	لایه‌ی AI Router داخلی () packages/ai با چند ارائه‌دهنده	هوش مصنوعی
نمونه‌خروجی‌ها، کاور، فاکتور	ذخیره‌ساز S3-سازگار () Cloudflare R2 یا معادل + next/image	فایل و تصویر
ایمیل تراکنشی و خبرنامه OTP و اطلاع‌رسانی فارسی	Resend یا (EN/AR) Postmark ارائه‌دهنده‌ی داخلی (کاوه‌نگار SMS.ir /	ایمیل پیامک
بدون کوکی سنگین؛ قیف و فیچرفلگ	Umami یا Plausible PostHog + (خودمیزبان) برای قیف	تحلیل
هشدار به تلگرام	Sentry + Uptime Kuma	خطا و آپتایم

چرا	انتخاب	لایه
زمان بندی خودکار پست در سه زبان	Postiz (متن باز، خودمیزبان) یا Buffer	شبکه های اجتماعی
کروم/فایرفاکس/اج با یک کد	WXT (فریم ورک افزونه چندمرورگری)	افزونه

۲. ساختار مخزن (Monorepo با pnpm + Turborepo)

prompt-market/	
├ apps/	
│ └ web/	# Next.js + Payload (پنل، سایت، API، Jobs)
│ │ └ app/[locale]/	# صفحات عمومی fa/en/ar
│ │ └ app/(payload)/	# پنل مدیریت
│ │ └ collections/	# Payload تعریف مدل داده‌ی (سند ۰۳)
│ │ └ jobs/	# پایپ لاین ها (سند ۰۴)
│ │ └ components/ui/	# CLI کپی شده با FarsiUI کامپوننت های
│ └ extension/	# (WXT) افزونه‌ی مرورگر
│ └ telegram-bot/	# API از طریق DB سبک، همان - (grammY) ربات
├ packages/	
│ └ ai/	# (judge)، قالب های سیستم، داور، AI Router،
├ شمارش هزینه	
│ └ search/	# رتبه بندی ترکیبی fa/ar، نرمال سازی
│ └ pricing/	# قیمت منطقه‌ای، گرد کردن روانی، شاخص گذاری تومان
│ └ i18n/	# پیام های رابط سه زبانه + تست کامل بودن ترجمه
│ └ config/	# eslint, tsconfig, prettier
├ infra/	# docker-compose، پیکربندی Coolify، پشتیبان گیری
├ docs/	# همین اسناد
└ .github/workflows/	# CI/CD و cron

۳. سه زبانه و جهت ها

مورد	fa	ar	en
dir	rtl	rtl	ltr
فونت	Vazirmatn (متن) / Estedad (تیترا)	Noto Naskh Arabic یا Markazi (متن) / Noto Kufi Arabic (تیترا)	Geist / Inter
ارقام	فارسی (۱۲۳)	قابل انتخاب: عربی- هندی (۱۲۳) یا لاتین — پیش فرض لاتین برای خلیج	لاتین
تقویم	شمسی (-day picker-persian)	میلادی (+ هجری در تاریخ های نمایشی)	میلادی
واحد پول	تومان	USD (نمایش تقریبی AED/SAR)	USD
ورود	OTP پیامکی	ایمیل + گوگل	ایمیل + گوگل

پیاده‌سازی: - app/[locale]/layout.tsx مقدار lang و dir روی <html> و DirectionProvider از FarsiUI را تنظیم می‌کند. - همه‌ی کلاس‌ها **منطقی** (ms-*, pe-*, *-start) — هیچ left/right فیزیکی (قانون lint). - تست CI: هر کلید پیام باید در هر سه زبان وجود داشته باشد؛ اسکرین‌شات Playwright از صفحات کلیدی در سه زبان.

۴. جست‌وجو

نرمال‌سازی (packages/search) قبل از ایندکس و قبل از کوئری: - فارسی: ی→ی، ک→کی، حذف اعراب و تطویل (-)، یکسان‌سازی نیم‌فاصله، ارقام فارسی/عربی → لاتین. - عربی: اِ اُ آ → ا، ة → ه (فقط در ایندکس کمکی)، ی → ی، حذف تشکیل.

رتبه‌بندی ترکیبی:

score = 0.45 · text_rank (برای غلط تایپی pg_trgm شباهت + ts_rank)
 + 0.35 · vector_sim (چندزبانه pgvector, embedding)
 + 0.20 · quality_score (سند ۰.۳)
 × locale_boost (اگر زبان محتوا = زبان کاربر ۱.۲)

- embedding چندزبانه یعنی جست‌وجوی فارسی، پرامپت عربی/انگلیسی مرتبط را هم پیدا می‌کند.
- هر جست‌وجوی **بدون نتیجه** ثبت می‌شود → ورودی «کارخانه‌ی محتوا» (سند ۰.۴).
- آستانه‌ی مهاجرت: اگر کاتالوگ < ۲۰۰ هزار رکورد یا > ۳۰۰ms p95 شد، Meilisearch اضافه می‌شود.

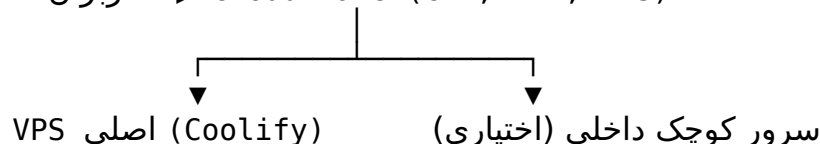
۵. پرداخت

نکته	روش	بازار
نیاز به اینماد و ثبت کسب‌وکار؛ اگر درگاه درخواست از IP خارج را رد کرد، یک سرویس کوچک callback روی سرور داخلی (مثلاً لیارا)	زرین‌پال (اصلی) + زیبال	FA
MoR مالیات/VAT را خودش حساب می‌کند؛ نیاز به نهاد حقوقی مجاز (سند ۰.۹)	Paddle یا Lemon Squeezy (Merchant of Record)	EN
پرداخت با Mada, Apple Pay, KNET	MoR بالا؛ در صورت نیاز بومی‌تر: Tap Payments / PayTabs	AR (خلیج)

یک جدول payments با فیلد provider؛ هر درگاه یک adapter با رابط یکسان (createCheckout, verify, refund, webhook).

۶. زیرساخت و استقرار

Cloudflare (CDN, WAF, DNS) → کاربران



- web (Next+Payload) ↳ callback ایرانی درگاه‌های
- PostgreSQL + pgvector
- Umami, Uptime Kuma, Postiz
- telegram-bot

(رمزنگاری شده، ۳۰ روز) R2 → پشتیبان‌گیری روزانه

- **Coolify** (PaaS خودمیزبان) استقرار با SSL، push، پشتیبان و لاگ را بدون دانش DevOps فراهم می‌کند.
- منابع شروع: یک VPS با ۴ GB RAM / vCPU کافی است.
- محیط‌ها: preview (هر PR) → staging - production (با تگ).
- نکته‌ی مهم: Next.js 16 + Payload 3 را در هفته‌ی ۱ با یک نمونه‌ی حداقلی تأیید کنید؛ اگر ناسازگاری بود، Next.js 15 با FarsiUI هم کار می‌کند.

V. CI/CD (GitHub Actions)

Workflow	تریگر	کار
ci.yml	هر PR	lint، typecheck، تست واحد، کامل بودن ترجمه، build
e2e.yml	هر PR	Playwright: خرید کامل در fa، ورود، جست‌وجو؛ اسکرین‌شات سه زبان
deploy.yml	push به main / تگ	استقرار staging / production از طریق webhook کولیفای
nightly.yml	cron شبانه	اجرای پایپ‌لاین‌هایی که نیاز به GitHub دارند (همگام‌سازی CC0)، گزارش وابستگی‌ها
lighthouse.yml	هفتگی	Core Web Vitals سه زبان؛ شکست اگر LCP > ۲۵s

۸. امنیت

- محتوای پرامپت پولی **هرگز** در HTML عمومی، JSON صفحه یا API بدون احراز هویت نمی‌آید (فقط پیش‌نمایش ناقص).
- واترمارک نامرئی متن (کاراکترهای عرض صفر با شناسه‌ی خریدار) برای ردیابی نشت.
- محدودیت نرخ (rate limit) روی جست‌وجو، OTP و اجرا؛ Turnstile روی فرم‌ها.
- اسرار در متغیرهای محیطی Coolify؛ هیچ کلیدی در مخزن.
- تأیید امضای webhook همه‌ی درگاه‌ها؛ idempotency روی سفارش.